

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ «МИФИ»

КАФЕДРА ФИНАНСОВОГО МОНИТОРИНГА (№75)

Лабораторная работа на тему:
Программная реализация задачи раздела
"Система аутентификации пользователей"

Выполнили студент:
Безносова Арина Юрьевна
Группа: С24-722
Проверил:
Курасов Юрий Викторович

Оценка: _____

Дата: _____

Подпись: _____

Москва, 2025

Выполнили: Безносова А.Б., Разинкина А.О.
Группа С24-722

Тема: программная реализация задачи раздела "Взаимодействие процессов, сигналы".

Инструменты разработки: компилятор языка C (GCC) и универсальный отладчик GNU Debugger (GDB) для ОС Linux Debian 12.

Задача

Разработать безопасную систему аутентификации, основанную на клиент-серверной архитектуре с выделенным демоном проверки прав доступа. Система должна обеспечивать безопасное взаимодействие веб-клиента, защищённого HTTPS-сервера, фонового демона и базы данных PostgreSQL. Основная цель — изолировать критичные операции проверки паролей в отдельном процессе и защитить данные на всех этапах передачи.

Теоретическая часть.

1. Введение

Разработанная система представляет собой безопасное **веб-приложение для аутентификации пользователей**, построенное по клиент-серверной архитектуре с разделением обязанностей. Основная цель системы — обеспечить защищённую проверку учетных данных пользователей, использующую **HTTPS-шифрование** для транспорта и **отдельный UNIX-демон (auth_daemon)** для обработки запросов на аутентификации, что повышает безопасность и изолирует доступ к базе данных.

Назначение системы:

- Предоставление пользователям защищённого веб-интерфейса (логин-формы) по HTTPS.
- Верификация паролей с использованием хеширования SHA-256.
- Разделение веб-сервера и логики проверки учётных данных для минимизации поверхности атаки.
- Хранение и проверка данных в СУБД PostgreSQL.

2. Общая архитектура системы

Система реализована на языке C и состоит из трёх основных компонентов, которые взаимодействуют через межпроцессное взаимодействие (IPC):

1. HTTPS-Сервер (server.c):

- Основной шлюз для клиентов (веб-браузеров).
- Обработывает входящие HTTPS-соединения на порту 8443.
- Обслуживает статические HTML-страницы (index.html, login.html, h404.html).
- Принимает POST-запросы с формы логина, вычисляет SHA-256 хеш пароля и перенаправляет запрос на демон аутентификации через **UNIX Domain Socket**.

2. Демон аутентификации (auth_daemon.c):

- Фоновый процесс (демон), работающий независимо от веб-сервера.
- Слушает UNIX Domain Socket по пути /tmp/auth_daemon.sock.
- Проверяет формат и корректность входных данных (имя пользователя и хеш).
- Выполняет запросы к базе данных через модуль database.c для проверки учётных данных.
- Возвращает веб-серверу простой текстовый результат (SUCCESS, USER_NOT_FOUND, INVALID_PASSWORD или коды ошибок).

3. Модуль работы с базой данных (database.c/database.h):

- Обеспечивает подключение к PostgreSQL (БД auth_system).
- Содержит функцию проверки соответствия хеша пароля пользователя (db_check_credentials).
- Используется **исключительно демоном аутентификации**, что концентрирует доступ к БД в одном процессе.

4. Алгоритм работы системы

Полный цикл обработки запроса аутентификации:

1. Клиент → HTTPS-сервер (HTTPS соединение)

- Пользователь заполняет форму входа в браузере
- Данные отправляются POST-запросом на /auth по HTTPS (порт 8443)
- Сервер устанавливает TLS-соединение с использованием OpenSSL

2. HTTPS-сервер → Демон (Unix-сокеты)

- Сервер парсит POST-данные (username и password)
- Вычисляет SHA-256 хэш пароля
- Формирует строку username:sha256_hash\n
- Подключается к Unix-сокету /tmp/auth_daemon.sock
- Отправляет запрос демону

3. Демон → База данных (через libpq)

- Демон принимает запрос через неблокирующий poll()
- Выполняет многоуровневую валидацию входных данных
- Подключается к PostgreSQL через модуль database.c
- Выполняет параметризованный SQL-запрос для проверки учётных данных

4. Обратный путь ответа

- База данных возвращает результат проверки
- Демон формирует ответ (SUCCESS/INVALID_PASSWORD/USER_NOT_FOUND/ERROR:*)
- Демон отправляет ответ серверу через Unix-сокеты
- Сервер формирует соответствующий HTTP-ответ (200/401)
- Клиент получает HTML-страницу с результатом аутентификации

Стандартные библиотеки C (общие для всех модулей):

- **stdio.h** – стандартный ввод/вывод (printf, fprintf, stderr, perror, scanf)
- **stdlib.h** – функции общего назначения (exit, malloc, free, atoi, NULL, EXIT_FAILURE)
- **string.h** – работа со строками (strcpy, strcmp, strlen, strchr, strncpy, memset)
- **ctype.h** – классификация символов (isalnum, isxdigit)
- **errno.h** – переменные и функции для обработки ошибок (errno, EINTR, EAGAIN, EWOULDBLOCK)

Системные библиотеки (POSIX/UNIX):

Для управления процессами и демонизации:

- **unistd.h** – POSIX-функции (close, read, write, sleep, fork, setsid, chdir, sysconf)
- **signal.h** – обработка сигналов (signal, sig_atomic_t)

- **sys/types.h** – системные типы данных (pid_t, size_t, ssize_t)

Для сетевого взаимодействия:

- **sys/socket.h** – функции для работы с сокетами (socket, bind, listen, accept, connect, send, recv, shutdown)
- **netinet/in.h** – структуры и константы для интернет-адресов (sockaddr_in, INADDR_ANY, htons)
- **arpa/inet.h** – преобразование адресов (inet_ntoa, inet_addr)
- **sys/un.h** – структуры для Unix-сокетов (sockaddr_un)

Для файловой системы и ввода-вывода:

- **sys/stat.h** – получение информации о файлах (stat, fstat, umask)
- **fcntl.h** – управление файловыми дескрипторами (fcntl, F_SETFL, F_GETFL, O_NONBLOCK, O_RDONLY, open)
- **sys/mman.h** – отображение файлов в память (mmap, munmap)

Для мультиплексирования:

- **poll.h** – мультиплексирование ввода-вывода (poll, struct pollfd, POLLIN, POLLERR, POLLHUP)

Сторонние библиотеки:

Для HTTPS-сервера (server.c):

- **openssl/ssl.h** – основные функции SSL/TLS для безопасных соединений
- **openssl/err.h** – обработка ошибок OpenSSL (ERR_print_errors_fp)
- **openssl/sha.h** – функции хеширования (SHA256)

Для работы с базой данных (auth_daemon.c, database.c):

- **postgresql/libpq-fe.h** – клиентская библиотека PostgreSQL (PQconnectdb, PQexecParams, PQgetvalue, PQclear, PQstatus, PQerrorMessage, PQfinish)

Собственные заголовочные файлы:

- **database.h** – интерфейс модуля работы с БД (объявляет функции db_connect и db_check_credentials)

СИСТЕМНЫЕ ВЫЗОВЫ И ФУНКЦИИ

Системные вызовы, используемые в системе:

Управление процессами и демонизация:

1. **fork()** – создание дочернего процесса (двойной fork для демонизации)
2. **setsid()** – создание новой сессии и отвязка от терминала
3. **umask()** – установка маски прав доступа для создаваемых файлов
4. **chdir()** – смена рабочего каталога на корневой (/)
5. **sysconf()** – получение системных параметров (_SC_OPEN_MAX – максимальное количество дескрипторов)
6. **exit()** – завершение процесса с указанным кодом возврата
7. **signal()** – установка обработчиков сигналов

Сетевые операции и сокеты:

8. **socket()** – создаёт конечную точку для сетевого обмена (TCP/UDP, Unix-сокет)
9. **bind()** – привязывает сокет к конкретному адресу и порту (или файлу Unix-сокета)
10. **listen()** – переводит сокет в режим прослушивания входящих подключений
11. **accept()** – принимает входящее соединение, создавая новый сокет для общения
12. **connect()** – устанавливает соединение с удалённым сокетом
13. **send()** – отправляет данные через сокет
14. **recv()** – принимает данные из сокета
15. **shutdown()** – корректно завершает соединение
16. **close()** – закрывает файловый дескриптор

Файловая система и память:

17. **stat()** – получает информацию о файле (размер, права доступа)
18. **open()** – открывает файл для чтения/записи
19. **mmap()** – отображает файл в память для быстрого доступа
20. **munmap()** – отменяет отображение файла из памяти
21. **unlink()** – удаление файла сокета при завершении
22. **fcntl()** – управление флагами файловых дескрипторов (установка неблокирующего режима)

Мультиплексирование:

23. **poll()** – ожидание событий на множестве дескрипторов с таймаутом

Функции работы со строками и памятью:

24. **memset()** – заполняет память указанным значением
25. **strlen(), strcmp(), strcpy(), sprintf(), snprintf(), scanf(), strchr()** – стандартные функции работы со строками
26. **perror()** – выводит сообщение об ошибке на основе errno

Криптографические обёртки (OpenSSL):

27. **SSL_accept(), SSL_read(), SSL_write(), SSL_shutdown()** – обёртки над системными вызовами для работы с TLS

Неявные вызовы (через libpq):

Библиотека libpq внутри себя использует системные вызовы для сетевого взаимодействия (socket, connect, send, recv, close), но на уровне модуля database.c они не вызываются напрямую.

Функции, реализованные в программе**В модуле server.c:**

1. **errExit()** – аварийное завершение с выводом ошибки
2. **create_server_socket()** – создание TCP-сокета сервера

3. **create_ssl_context(), configure_ssl_context()** – настройка SSL/TLS
4. **handle_client()** – обработка подключения клиента
5. **parse_auth_data()** – разбор POST-данных аутентификации
6. **sha256_hash()** – хеширование SHA-256
7. **handle_auth_request()** – обработка запроса аутентификации
8. **serve_static_file()** – отправка статических файлов

В модуле **auth_daemon.c**:

1. **errExit()** – аварийное завершение с выводом ошибки (макрос)
2. **term_handler()** – обработчик сигналов завершения (SIGTERM, SIGINT)
3. **skeleton_daemon()** – превращение процесса в демона
4. **is_valid_sha256_hash()** – проверка формата SHA-256 хэша (64 hex символа)
5. **is_valid_username()** – валидация имени пользователя (длина и допустимые символы)
6. **process_auth_request()** – обработка запроса аутентификации (разбор, валидация, проверка в БД)
7. **run_daemon()** – основной цикл работы демона с использованием poll()
8. **main()** – точка входа в программу

В модуле **database.c**:

1. **db_connect()** – установка и поддержание соединения с базой данных
2. **db_check_credentials()** – проверка логина и хэша пароля в базе данных

Алгоритм серверной части программы:

1. Сервер запускается на порту 8443 (HTTPS). Загружаются SSL-сертификат (server.crt) и приватный ключ (server.key). Используется OpenSSL для шифрования соединений.
2. Сервер создаёт сокет, привязывает его к порту и переходит в режим прослушивания. При подключении клиента устанавливается TLS-соединение.
3. После установки безопасного соединения сервер читает HTTP-запрос от клиента. Первым этапом является парсинг входящих данных, в ходе которого извлекаются ключевые элементы запроса: HTTP-метод (GET или POST), целевой путь (например, /, /login.html или /auth) и, для POST-запросов, тело с данными формы.

Если клиент запрашивает статический контент методом GET, сервер сопоставляет путь с файлами в корневой директории /html. При запросе корневого пути / клиенту автоматически отправляется файл index.html. Для других путей, например /login.html, сервер пытается найти и отправить соответствующий файл. Если запрашиваемый ресурс отсутствует, пользователю возвращается специальная страница h404.html с HTTP-статусом 404 Not Found.

Для обработки данных аутентификации сервер принимает POST-запросы только по пути /auth. После извлечения полей username и password из тела запроса пароль хешируется алгоритмом SHA-256.

4. Сервер подключается к Unix-сокету /tmp/auth_daemon.sock. Отправляет демону строку в формате логин:хэш_пароля. Получает от демона один из ответов:

SUCCESS - аутентификация успешна.

INVALID_PASSWORD - неверный пароль.

USER_NOT_FOUND - пользователь не существует.

- После обработки ответа демона сервер формирует HTTP-ответ. При успешной аутентификации отправляется страница с подтверждением и ссылкой на главную. В случае ошибок возвращаются соответствующие статусы: 400 при некорректных данных, 401 при неудачной проверке логина/пароля, 404 если файл не найден. Все данные передаются через защищённое SSL-соединение с корректными HTTP-заголовками.
- После отправки ответа соединение с клиентом корректно закрывается, сервер возвращается в режим ожидания нового подключения.

Алгоритм работы модуля базы данных:

- Модуль database.c отвечает за взаимодействие с PostgreSQL для хранения и проверки учётных данных пользователей. Он предоставляет серверу или демону аутентификации интерфейс для проверки паролей без необходимости прямого доступа к файловой системе или таблицам БД.
- При первом обращении модуль устанавливает соединение с базой данных auth_system, используя параметры подключения по умолчанию. Соединение кэшируется в статической переменной для повторного использования.
- Для проверки учётных данных выполняется параметризованный SQL-запрос:

```
sql
SELECT password_hash FROM users WHERE username = $1
```

 Использование параметров предотвращает SQL-инъекции.
- Если пользователь не найден, функция возвращает 0. Если найден, хеш пароля, полученный от клиента (после SHA-256), сравнивается с хешом, хранящимся в базе данных.
- Результат проверки возвращается вызывающему коду: 1 — аутентификация успешна, 0 — неудача (пользователь не найден или пароль неверен).

Алгоритм работы демона аутентификации:

- Демон запускается как фоновый процесс (daemon) с полной изоляцией от терминала и родительского окружения. Используется двойной fork() для гарантии, что процесс не станет лидером сессии и не сможет получить управляющий терминал.
- Демон создаёт UNIX-сокеты по пути /tmp/auth_daemon.sock и переводит его в неблокирующий режим. Сокет работает в потоковом режиме (SOCK_STREAM), обеспечивая надёжную двустороннюю связь с HTTPS-сервером.
- Для обработки множества одновременных подключений используется системный вызов poll(), который отслеживает события на 11 файловых дескрипторах (10 клиентов + 1 серверный сокет). Демон поддерживает до 10 параллельных подключений.
- При получении нового подключения демон принимает его через accept(), находит свободный слот в массиве клиентов, сохраняет дескриптор и настраивает неблокирующий режим для данного соединения.
- Данные от клиентов читаются асинхронно. Демон накапливает поступающие данные в буфере каждого клиента до получения символа новой строки \n, который обозначает конец запроса. Формат запроса: `username:sha256_hash\n`.
- При получении полного запроса вызывается функция process_auth_request(), которая выполняет многоуровневую валидацию:
 - Проверка наличия разделителя :
 - Валидация имени пользователя (длина, допустимые символы)
 - Валидация хэша пароля (длина 64 символа, hex-формат)
- После успешной валидации демон подключается к базе данных PostgreSQL через библиотеку libpq и выполняет два запроса:
 - Проверка существования пользователя (SELECT id FROM users WHERE username = \$1)
 - Сравнение переданного хэша с сохранённым в базе (через функцию db_check_credentials)

8. В зависимости от результата проверки демон отправляет один из ответов:
 - SUCCESS\n — аутентификация успешна
 - INVALID_PASSWORD\n — неверный пароль
 - USER_NOT_FOUND\n — пользователь не существует
 - ERROR:тип_ошибки\n — ошибка валидации или работы с БД
9. После отправки ответа буфер клиента очищается, соединение остаётся открытым для возможных последующих запросов. Демон продолжает работу в бесконечном цикле до получения сигнала завершения.
10. При корректном завершении (по сигналам SIGTERM или SIGINT) демон закрывает все клиентские соединения, серверный сокет и удаляет файл сокета.

Протокол взаимодействия с HTTPS-сервером:

- **Запрос** от сервера: username:sha256_hash\n (например: alice:ef92b778...94f\n)

- **Ответы** демона: SUCCESS\n, INVALID_PASSWORD\n, USER_NOT_FOUND\n, ERROR:bad_format\n, ERROR:db_connection\n и другие.

Структуры данных:

- struct Client — хранит состояние каждого подключённого клиента (дескриптор сокета, буфер данных, текущая длина буфера)
- struct pollfd — используется для мультиплексирования ввода-вывода через poll()
- struct sockaddr_un — адрес UNIX-сокета
- PGconn, PGresult — структуры библиотеки libpq для работы с PostgreSQL

Практическая часть.**Серверная часть Server.c.****1. Вспомогательные функции****Переменные:**

- hash[SHA256_DIGEST_LENGTH] - временный массив для хранения хеша
- output - результирующая строка (65 символов: 64 hex + ноль-терминатор)

Описание:

- errExit - завершение программы с выводом ошибки
- sha256_hash - вычисление SHA-256 хеша строки, результат в hex-формате

```
void errExit(const char* msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

void sha256_hash(const char* input, char* output) {
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)input, strlen(input), hash);
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++) {
        sprintf(output + i * 2, "%02x", hash[i]);
    }
    output[SHA256_DIGEST_LENGTH * 2] = '\0';
}
```

2. Создание серверного сокета**Переменные:**

- server_sock - дескриптор серверного сокета
- server_addr - структура адреса сервера
- opt - флаг для setsockopt

Описание: Создание и настройка TCP-сокета сервера:

1. Создание сокета (socket)
2. Установка опции переиспользования адреса (SO_REUSEADDR)
3. Настройка адреса сервера (все интерфейсы, указанный порт)
4. Привязка сокета к адресу (bind)
5. Перевод в режим прослушивания (listen)

```
int create_server_socket(int port) {
    int server_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (server_sock < 0) errExit("socket");

    int opt = 1;
    setsockopt(server_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(port);

    if (bind(server_sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        close(server_sock);
    }
}
```

```

errExit("bind");
}

if (listen(server_sock, 10) < 0) {
close(server_sock);
errExit("listen");
}

return server_sock;
}

```

3. Настройка SSL/TLS

Переменные:

- ctx - SSL контекст (хранит настройки SSL)
- method - метод TLS для сервера

Описание: Инициализация и конфигурация SSL/TLS контекста:

- create_ssl_context - создание SSL-контекста для сервера
- configure_ssl_context - загрузка сертификата и приватного ключа, проверка их соответствия

```

SSL_CTX* create_ssl_context() {
const SSL_METHOD* method = TLS_server_method();
if (!method) {
ERR_print_errors_fp(stderr);
exit(EXIT_FAILURE);
}

SSL_CTX* ctx = SSL_CTX_new(method);
if (!ctx) {
fprintf(stderr, "Unable to create SSL context\n");
ERR_print_errors_fp(stderr);
exit(EXIT_FAILURE);
}

return ctx;
}

void configure_ssl_context(SSL_CTX* ctx) {
if (SSL_CTX_use_certificate_file(ctx, "server.crt", SSL_FILETYPE_PEM) <= 0) {
fprintf(stderr, "Failed to load certificate\n");
ERR_print_errors_fp(stderr);
exit(EXIT_FAILURE);
}

if (SSL_CTX_use_PrivateKey_file(ctx, "server.key", SSL_FILETYPE_PEM) <= 0) {
fprintf(stderr, "Failed to load private key\n");
ERR_print_errors_fp(stderr);
exit(EXIT_FAILURE);
}

if (!SSL_CTX_check_private_key(ctx)) {
fprintf(stderr, "Private key does not match certificate!\n");
exit(EXIT_FAILURE);
}

printf("SSL certificate loaded successfully\n");
}

```

4. Парсинг данных аутентификации

Переменные:

- user_field, pass_field - имена полей из запроса
- username, password - извлеченные значения (будут заполнены)

Описание: Извлечение данных из POST-запроса аутентификации:

1. Парсинг строки формата `username=value&password=value`
2. Проверка правильности имен полей
3. Простой URL decode (замена '+' на пробелы)

```
int parse_auth_data(const char* post_data, char* username, char* password) {
    char user_field[32], pass_field[32];

    if (sscanf(post_data, "%31[^]=%63[^\&]&%31[^]=%63s",
        user_field, username,
        pass_field, password) != 4) {
        printf("Error parsing POST data: %s\n", post_data);
        return 0;
    }

    if (strcmp(user_field, "username") != 0 ||
        strcmp(pass_field, "password") != 0) {
        printf("Invalid field names in POST data\n");
        return 0;
    }
    for (char* p = username; *p; p++) if (*p == '+') *p = ' ';
    for (char* p = password; *p; p++) if (*p == '+') *p = ' ';

    printf("Parsing successful: user='%s', pass='%s'\n", username, password);
    return 1;
}
```

5. Обработка запроса аутентификации

Переменные:

- ssl - SSL-соединение с клиентом
- password_hash[65] - хеш пароля в hex-формате
- sock - сокет для подключения к демону
- addr - адрес Unix-сокета
- daemon_response[32] - ответ от демона

Описание: Полная обработка запроса на аутентификацию:

1. Парсинг данных из POST
2. Хеширование пароля SHA-256
3. Подключение к Unix-сокету демона аутентификации
4. Отправка данных в формате `username:password\_hash`
5. Обработка ответа демона и отправка соответствующего HTTP-ответа клиенту

```
void handle_auth_request(SSL* ssl, char* post_data) {
    char username[MAX_USERNAME];
    char password[MAX_PASSWORD];
    char response[BUFFER_SIZE];
```

```

printf("Authentication attempt\n");

if (!parse_auth_data(post_data, username, password)) {
    snprintf(response, sizeof(response),
        "HTTP/1.1 400 Bad Request\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 55\r\n\r\n"
        "<h1>Data error</h1><a href='/'>Back</a>");
    SSL_write(ssl, response, strlen(response));
    return;
}
int sock;
struct sockaddr_un addr;
char request[256];
char daemon_response[32];
char password_hash[65];

sha256_hash(password, password_hash);
printf("[server] Password hash: %s\n", password_hash);

snprintf(request, sizeof(request), "%s:%s\n", username, password_hash);

sock = socket(AF_UNIX, SOCK_STREAM, 0);
if (sock == -1) {
    perror("[server] socket");
    goto auth_error;
}

struct timeval timeout;
timeout.tv_sec = 5;
timeout.tv_usec = 0;
setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));
setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, &timeout, sizeof(timeout));

memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strncpy(addr.sun_path, UNIX_SOCKET_PATH, sizeof(addr.sun_path) - 1);

if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
    printf("[server] Cannot connect to auth daemon\n");
    close(sock);
    goto auth_error;
}

ssize_t sent_bytes = send(sock, request, strlen(request), 0);
if (sent_bytes == -1) {
    perror("[server] send to auth daemon");
    close(sock);
    goto auth_error;
}

memset(daemon_response, 0, sizeof(daemon_response));
ssize_t bytes = recv(sock, daemon_response, sizeof(daemon_response) - 1, 0);
close(sock);

if (bytes > 0) {
    daemon_response[bytes] = '\0';
    printf("[server] Auth daemon response: %s", daemon_response);
}

```

```

if (strcmp(daemon_response, "SUCCESS\n") == 0) {
    snprintf(response, sizeof(response),
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 45\r\n\r\n"
        "<h1>Login successful!</h1><a href='/'>Home</a>");
}
else if (strcmp(daemon_response, "INVALID_PASSWORD\n") == 0) {
    snprintf(response, sizeof(response),
        "HTTP/1.1 401 Unauthorized\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 65\r\n\r\n"
        "<h1>Invalid password</h1><p>Password is incorrect</p><a href='/'>Back</a>");
}
else if (strcmp(daemon_response, "USER_NOT_FOUND\n") == 0) {
    snprintf(response, sizeof(response),
        "HTTP/1.1 401 Unauthorized\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 85\r\n\r\n"
        "<h1>User not found</h1><p>User '%s' does not exist</p><a href='/'>Back</a>",
        username);
}
else {
    snprintf(response, sizeof(response),
        "HTTP/1.1 401 Unauthorized\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 55\r\n\r\n"
        "<h1>Authentication failed</h1><a href='/'>Back</a>");
}
}
else {
    goto auth_error;
}

SSL_write(ssl, response, strlen(response));
return;

auth_error:
    snprintf(response, sizeof(response),
        "HTTP/1.1 401 Unauthorized\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 55\r\n\r\n"
        "<h1>Authentication error</h1><a href='/'>Back</a>");
    SSL_write(ssl, response, strlen(response));
}

```

6. Отправка статических файлов

Переменные:

- full_path[512] - полный путь к файлу
- fs - структура с информацией о файле
- fd - файловый дескриптор
- fmap - указатель на отображение файла в память

Описание: Отправка HTML/CSS/JS файлов клиенту:

1. Построение полного пути к файлу
2. Проверка существования файла

3. Если файл не найден - отправка страницы 404
4. Использование mmap для эффективной отправки файлов
5. Формирование правильных HTTP-заголовков

```
void serve_static_file(SSL* ssl, const char* path) {
    char full_path[512];
    snprintf(full_path, sizeof(full_path), "%s%s", WROOT, path);

    struct stat fs;
    if (stat(full_path, &fs) < 0) {
        char error_path[512];
        snprintf(error_path, sizeof(error_path), "%s%s", WROOT, H404);

        int fd = open(error_path, O_RDONLY);
        if (fd >= 0 && stat(error_path, &fs) == 0) {
            char headers[512];
            snprintf(headers, sizeof(headers),
                "%sContent-Length: %ld\r\n\r\n", ERR404, fs.st_size);

            SSL_write(ssl, headers, strlen(headers));
            void* fmap = mmap(NULL, fs.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
            if (fmap != (void*)-1) {
                SSL_write(ssl, fmap, fs.st_size);
                munmap(fmap, fs.st_size);
            }
            close(fd);
        }
        return;
    }

    int fd = open(full_path, O_RDONLY);
    if (fd >= 0) {
        char headers[512];
        snprintf(headers, sizeof(headers),
            "%sContent-Length: %ld\r\n\r\n", HTTPS_HEADER, fs.st_size);

        SSL_write(ssl, headers, strlen(headers));
        void* fmap = mmap(NULL, fs.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
        if (fmap != (void*)-1) {
            SSL_write(ssl, fmap, fs.st_size);
            munmap(fmap, fs.st_size);
        }
        close(fd);
        printf("File sent: %s (%ld bytes)\n", path, fs.st_size);
    }
}
```

7. Обработка клиентского подключения

Переменные:

- ssl - SSL-соединение
- buffer[BUFFER_SIZE] - буфер для HTTP-запроса
- method[16], path[256] - извлеченные метод и путь
- body - указатель на тело POST-запроса

Описание: Основная функция обработки клиента:

1. Установка SSL-соединения

2. Чтение HTTP-запроса
3. Определение метода и пути
4. Маршрутизация:
 - GET / → index.html
 - GET /file → статический файл
 - POST /auth → аутентификация
5. Закрытие соединения

```

void handle_client(int client_sock, SSL_CTX* ctx) {
    SSL* ssl = SSL_new(ctx);
    SSL_set_fd(ssl, client_sock);

    if (SSL_accept(ssl) <= 0) {
        ERR_print_errors_fp(stderr);
        SSL_free(ssl);
        close(client_sock);
        return;
    }

    char buffer[BUFFER_SIZE];
    int bytes = SSL_read(ssl, buffer, sizeof(buffer) - 1);
    if (bytes <= 0) {
        SSL_free(ssl);
        close(client_sock);
        return;
    }
    buffer[bytes] = '\0';

    char method[16], path[256];
    if (sscanf(buffer, "%15s %255s", method, path) != 2) {
        SSL_free(ssl);
        close(client_sock);
        return;
    }

    printf("%s %s\n", method, path);

    if (strcmp(method, GET) == 0) {
        if (strcmp(path, "/") == 0) {
            serve_static_file(ssl, INDEX);
        }
        else {
            serve_static_file(ssl, path);
        }
    }
    else if (strcmp(method, POST) == 0 && strcmp(path, "/auth") == 0) {
        char* body = strstr(buffer, "\r\n\r\n");
        if (body) {
            body += 4;
            handle_auth_request(ssl, body);
        }
        else {
            char response[] = "HTTP/1.1 400 Bad Request\r\n\r\n";
            SSL_write(ssl, response, strlen(response));
        }
    }
    else {

```

```

char response[] = "HTTP/1.1 405 Method Not Allowed\r\n\r\n";
SSL_write(ssl, response, strlen(response));
}

SSL_shutdown(ssl);
SSL_free(ssl);
}

```

8. Основная функция

Переменные:

- ctx - SSL контекст
- server_sock - серверный сокет
- client_sock - клиентский сокет
- client_addr - адрес клиента

Описание:

1. Вывод информации о конфигурации
2. Инициализация OpenSSL
3. Создание SSL-контекста и серверного сокета
4. Бесконечный цикл принятия подключений
5. Обработка каждого клиента в handle_client
6. Очистка ресурсов при завершении

```

int main() {
printf("Starting HTTPS server (port %d)\n", SERVER_PORT);
printf("HTML directory: %s\n", WROOT);
printf("SSL certificate: server.crt\n");
printf("SSL private key: server.key\n");
printf("Auth daemon socket: %s\n", UNIX_SOCKET_PATH);
printf("Open: https://localhost:%d\n", SERVER_PORT);
printf("\n");

SSL_library_init();
SSL_load_error_strings();
OpenSSL_add_all_algorithms();

SSL_CTX* ctx = create_ssl_context();
configure_ssl_context(ctx);

int server_sock = create_server_socket(SERVER_PORT);
printf("Server ready for HTTPS connections\n\n");

while (1) {
struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);
int client_sock = accept(server_sock, (struct sockaddr*)&client_addr,
&client_len);

if (client_sock >= 0) {
printf("New connection from %s\n", inet_ntoa(client_addr.sin_addr));
handle_client(client_sock, ctx);
close(client_sock);
printf("Connection closed\n\n");
}
}

SSL_CTX_free(ctx);

```

```
close(server_sock);
return 0;
}
```

Демон аутентификации auth_daemon.

1. Макрос аварийного завершения

Описание: Макрос для обработки критических ошибок. Выводит сообщение об ошибке через perror() и завершает программу с кодом EXIT_FAILURE.

```
#define errExit(msg) do { perror(msg); exit(EXIT_FAILURE); } while (0)
```

2. Обработчик сигналов завершения

Переменные:

sig — номер полученного сигнала (не используется, подавление warning через (void)sig)

Описание: Устанавливает флаг stop_flag = 1 при получении сигналов SIGTERM или SIGINT. Флаг проверяется в основном цикле run_daemon(), что приводит к корректному завершению с закрытием всех ресурсов.

```
static void term_handler(int sig) {
    (void)sig;
    stop_flag = 1;
}
```

3. Демонизация процесса

Переменные:

- pid — идентификатор процесса для управления fork()
 - x — счётчик для закрытия файловых дескрипторов
- Описание:** Функция skeleton_daemon() превращает процесс в фоновый демон:

1. Первый fork() создаёт дочерний процесс, родительский завершается.
2. setsid() создаёт новую сессию, отвязывая процесс от терминала.
3. Устанавливаются обработчики сигналов:
SIGCHLD, SIGHUP — игнорируются
SIGTERM, SIGINT — обрабатываются через term_handler() для корректного завершения
SIGPIPE — игнорируется для контроля ошибок записи через errno
4. Второй fork() гарантирует, что процесс не сможет стать лидером сессии.
5. umask(0) сбрасывает маску прав доступа.
6. chdir("/") переходит в корневую директорию, чтобы не блокировать монтирование файловых систем.
7. Закрываются все унаследованные файловые дескрипторы через цикл от sysconf(_SC_OPEN_MAX) до 0.

```
static void skeleton_daemon()
{
    pid_t pid;

    pid = fork();
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS);

    if (setsid() < 0) exit(EXIT_FAILURE);

    signal(SIGCHLD, SIG_IGN);
    signal(SIGHUP, SIG_IGN);
```

```

signal(SIGTERM, term_handler);
signal(SIGINT, term_handler);
signal(SIGPIPE, SIG_IGN);

pid = fork();
if (pid < 0) exit(EXIT_FAILURE);
if (pid > 0) exit(EXIT_SUCCESS);

umask(0);
chdir("/");

int x;
for (x = sysconf(_SC_OPEN_MAX); x >= 0; x--) {
    close(x);
}
}

```

4. Валидация SHA-256 хэша

Переменные:

- str — входная строка для проверки

Описание: Проверяет, что строка является корректным SHA-256 хэшем:

1. Длина должна быть ровно 64 символа (MIN_HASH_LEN).
2. Все символы должны быть шестнадцатеричными цифрами (isxdigit()).

```

static int is_valid_sha256_hash(const char* str) {
    if (strlen(str) != MIN_HASH_LEN) return 0;

    for (int i = 0; i < MIN_HASH_LEN; i++) {
        if (!isxdigit(str[i])) return 0;
    }
    return 1;
}

```

5. Валидация имени пользователя

Переменные:

- str — имя пользователя для проверки
- len — длина строки

Описание: Валидирует имя пользователя по критериям:

1. Длина от 1 до 49 символов (MIN_USERNAME_LEN и USERNAME_MAX).
2. Допустимые символы: буквы, цифры, _, -, @, . (isalnum() и проверка спецсимволов).
3. Запрещены пробелы, специальные и управляющие символы.

```

static int is_valid_username(const char* str) {
    int len = strlen(str);
    if (len < MIN_USERNAME_LEN || len >= USERNAME_MAX) return 0;

    for (int i = 0; i < len; i++) {
        if (!isalnum(str[i]) && str[i] != '_' && str[i] != '-' &&
            str[i] != '@' && str[i] != '.') {
            return 0;
        }
    }
    return 1;
}

```

6. Обработка запроса аутентификации

Переменные:

- username[USERNAME_MAX] — буфер для имени пользователя
- password_hash[HASH_MAX] — буфер для хэша пароля
- response[BUF_SIZE] — буфер для ответа клиенту
- auth_result — результат проверки в БД
- colon — указатель на разделитель : в запросе
- hash_start — указатель на начало хэша
- newline — указатель на символ \n
- conn — соединение с БД PostgreSQL
- user_res — результат SQL-запроса

Описание: Выполняет полный цикл обработки запроса:

1. Разбор запроса: поиск разделителя : (strchr) и символа \n.
2. Валидация username: проверка длины, формата через is_valid_username(), копирование в буфер с защитой от переполнения (strncpy).
3. Валидация password_hash: проверка длины (64 символа), hex-формата через is_valid_sha256_hash().
4. Подключение к БД: вызов db_connect() для получения соединения.
5. Проверка пользователя: параметризованный SQL-запрос на существование username (PQexecParams).
6. Проверка пароля: вызов db_check_credentials() для сравнения хэшей.
7. Формирование ответа: SUCCESS, INVALID_PASSWORD, USER_NOT_FOUND или ERROR:* в зависимости от результата.
8. Отправка ответа: отправка результата через send() и очистка буфера клиента (buf_len = 0, memset).

```
static void process_auth_request(struct Client* client)
{
    char username[USERNAME_MAX];
    char password_hash[HASH_MAX];
    char response[BUF_SIZE];
    int auth_result;

    char* colon = strchr(client->buf, ':');
    if (colon == NULL) {
        strcpy(response, "ERROR:bad_format\n");
        goto send_response;
    }

    *colon = '\0';
    size_t username_len = strlen(client->buf);

    if (username_len == 0) {
        strcpy(response, "ERROR:empty_username\n");
        goto send_response;
    }

    if (username_len >= USERNAME_MAX) {
        strcpy(response, "ERROR:username_too_long\n");
        goto send_response;
    }

    if (!is_valid_username(client->buf)) {
        strcpy(response, "ERROR:invalid_username\n");
        goto send_response;
    }

    strncpy(username, client->buf, sizeof(username) - 1);
    username[sizeof(username) - 1] = '\0';
```

```

char* hash_start = colon + 1;
char* newline = strchr(hash_start, '\n');
if (newline == NULL) {
    strcpy(response, "ERROR:no_newline\n");
    goto send_response;
}

size_t hash_len = newline - hash_start;

if (hash_len == 0) {
    strcpy(response, "ERROR:empty_hash\n");
    goto send_response;
}

if (hash_len >= HASH_MAX) {
    strcpy(response, "ERROR:hash_too_long\n");
    goto send_response;
}

if (hash_len < MIN_HASH_LEN) {
    strcpy(response, "ERROR:hash_too_short\n");
    goto send_response;
}

strncpy(password_hash, hash_start, hash_len);
password_hash[hash_len] = '\0';

if (!is_valid_sha256_hash(password_hash)) {
    strcpy(response, "ERROR:invalid_hash_format\n");
    goto send_response;
}

PGconn *conn = db_connect();
if (!conn) {
    strcpy(response, "ERROR:db_connection\n");
    goto send_response;
}

const char *check_user_query = "SELECT id FROM users WHERE username = $1";
const char *check_user_params[1] = { username };

PGresult *user_res = PQexecParams(conn, check_user_query, 1, NULL,
check_user_params, NULL, NULL, 0);

if (PQresultStatus(user_res) != PGRES_TUPLES_OK) {
    PQclear(user_res);
    strcpy(response, "ERROR:db_query\n");
    goto send_response;
}

if (PQntuples(user_res) == 0) {
    PQclear(user_res);
    strcpy(response, "USER_NOT_FOUND\n");
    goto send_response;
}

PQclear(user_res);

```

```

    auth_result = db_check_credentials(username, password_hash);

    if (auth_result) {
        strcpy(response, "SUCCESS\n");
    }
    else {
        strcpy(response, "INVALID_PASSWORD\n");
    }

send_response:
    send(client->fd, response, strlen(response), 0);

    client->buf_len = 0;
    memset(client->buf, 0, sizeof(client->buf));
}

```

7. Основной цикл демона

Переменные:

- server_fd — дескриптор серверного сокета
- clients[MAX_CLIENTS] — массив структур Client
- pfdс[MAX_CLIENTS + 1] — массив struct pollfd для мультиплексирования
- self — struct sockaddr_un адреса серверного сокета
- ready — количество готовых дескрипторов от poll()
- client_fd — дескриптор клиентского сокета
- slot — индекс свободного слота в массиве клиентов
- bytes_read — количество прочитанных байт

Описание: Реализует основной цикл работы демона:

1. Инициализация: обнуление структур (memset), создание сокета (socket), привязка (bind), прослушивание (listen).
2. Настройка неблокирующего режима: fcntl с O_NONBLOCK для серверного сокета.
3. Основной цикл poll(): while (!stop_flag) с ожиданием событий (poll) с таймаутом 1000 мс.
4. Обработка новых подключений: при событии POLLIN на серверном сокете вызывается accept(), находится свободный слот, настраивается неблокирующий режим для клиента.
5. Обработка клиентских данных: при событии POLLIN на клиентском сокете данные читаются через recv(), накапливаются в буфере до символа \n, затем вызывается process_auth_request().
6. Обработка ошибок: закрытие соединений при POLLERR, POLLHUP или ошибках чтения.
7. Корректное завершение: при установке stop_flag закрываются все соединения и удаляется файл сокета (unlink).

```

static void run_daemon()
{
    int server_fd, rc;
    socklen_t len;
    struct sockaddr_un self;
    struct Client clients[MAX_CLIENTS];
    struct pollfd pfdс[MAX_CLIENTS + 1];

    memset(&self, 0, sizeof(self));
    memset(clients, 0, sizeof(clients));

    for (int i = 0; i < MAX_CLIENTS; i++) {
        pfdс[i].fd = -1;
        pfdс[i].events = 0;
    }
}

```



```

    clients[i].fd = -1;
}

len = sizeof(self);

unlink(SOCKET_PATH);
if ((server_fd = socket(AF_UNIX, SOCK_STREAM, 0)) == -1)
    errExit("socket");

self.sun_family = AF_UNIX;
strcpy(self.sun_path, SOCKET_PATH);

if ((rc = bind(server_fd, (struct sockaddr*)&self, len)) == -1)
    errExit("bind");

if ((listen(server_fd, 10)) == -1)
    errExit("listen");

fcntl(server_fd, F_SETFL, fcntl(server_fd, F_GETFL) | O_NONBLOCK);

pfds[MAX_CLIENTS].fd = server_fd;
pfds[MAX_CLIENTS].events = POLLIN;

while (!stop_flag) {
    int ready = poll(pfds, MAX_CLIENTS + 1, 1000);

    if (ready == -1) {
        if (errno == EINTR) continue;
        errExit("poll");
    }

    if (ready == 0) continue;

    if (pfds[MAX_CLIENTS].revents & POLLIN) {
        int client_fd;
        struct sockaddr_un peer_addr;
        socklen_t peer_len = sizeof(peer_addr);

        client_fd = accept(server_fd, (struct sockaddr*)&peer_addr,
&peer_len);

        if (client_fd == -1) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) continue;
            errExit("accept");
        }

        int slot = -1;
        for (int i = 0; i < MAX_CLIENTS; i++) {
            if (clients[i].fd == -1) {
                slot = i;
                break;
            }
        }

        if (slot != -1) {
            clients[slot].fd = client_fd;
            clients[slot].buf_len = 0;
            memset(clients[slot].buf, 0, sizeof(clients[slot].buf));

```

```

        fcntl(client_fd, F_SETFL, fcntl(client_fd, F_GETFL) | O_NONBLOCK);
        pfds[slot].fd = client_fd;
        pfds[slot].events = POLLIN;
    }
    else {
        close(client_fd);
    }
}

for (int i = 0; i < MAX_CLIENTS; i++) {
    if (pfds[i].fd == -1) continue;

    if (pfds[i].revents & (POLLERR | POLLHUP)) {
        close(clients[i].fd);
        clients[i].fd = -1;
        pfds[i].fd = -1;
        continue;
    }

    if (pfds[i].revents & POLLIN) {
        ssize_t bytes_read = recv(clients[i].fd,
            clients[i].buf + clients[i].buf_len,
            sizeof(clients[i].buf) - clients[i].buf_len - 1,
            0);

        if (bytes_read > 0) {
            clients[i].buf_len += bytes_read;
            clients[i].buf[clients[i].buf_len] = '\0';

            if (strchr(clients[i].buf, '\n') != NULL) {
                process_auth_request(&clients[i]);
            }
        }
        else if (bytes_read == 0) {
            close(clients[i].fd);
            clients[i].fd = -1;
            pfds[i].fd = -1;
        }
        else if (bytes_read == -1 && errno != EAGAIN && errno !=
EWOULDBLOCK) {
            close(clients[i].fd);
            clients[i].fd = -1;
            pfds[i].fd = -1;
        }
    }
}

for (int i = 0; i < MAX_CLIENTS; i++) {
    if (clients[i].fd != -1) close(clients[i].fd);
}

close(server_fd);
unlink(SOCKET_PATH);
}

```

8. Точка входа в программу

Описание: Запускает демона:

1. Вызов `skeleton_daemon()` для превращения в фоновый процесс.
2. Вызов `run_daemon()` для запуска основного цикла работы.
3. Возврат `EXIT_SUCCESS` по завершении.

```
int main(void)
{
    skeleton_daemon();
    run_daemon();
    return EXIT_SUCCESS;
}
```

Модуль базы данных `database.h` + `database.c`

1. Функция `db_connect()`

Переменные:

- `db_conn` — статический указатель на объект соединения с БД (`PGconn`).
- `conninfo` — строка параметров подключения к БД.

Описание: Управляет подключением к базе данных PostgreSQL. Реализует простое кэширование: если соединение уже установлено и активно, оно возвращается немедленно. В противном случае создаётся новое подключение. При ошибке подключения информация выводится в `stderr`, соединение закрывается, а функция возвращает `NULL`.

```
PGconn* db_connect() {
    if (db_conn != NULL && PQstatus(db_conn) == CONNECTION_OK) {
        return db_conn;
    }
    const char *conninfo = "dbname=auth_system";
    db_conn = PQconnectdb(conninfo);
    if (PQstatus(db_conn) != CONNECTION_OK) {
        fprintf(stderr, "Database connection failed: %s\n",
PQerrorMessage(db_conn));
        PQfinish(db_conn);
        db_conn = NULL;
        return NULL;
    }
    return db_conn;
}
```

2. Функция `db_check_credentials()`

Переменные:

- `conn` — указатель на активное соединение с БД.
- `query` — текст параметризованного SQL-запроса.
- `params` — массив параметров для запроса (в данном случае один элемент: имя пользователя).
- `res` — указатель на результат выполнения запроса (`PGresult`).
- `stored_hash` — хеш пароля, полученный из базы данных.

Описание: Выполняет проверку учётных данных. Сначала получает соединение с БД. Затем выполняет параметризованный запрос на поиск хеша пароля по имени пользователя. Если запрос выполнялся успешно и вернул хотя бы одну строку, извлекает сохранённый хеш и сравнивает его с переданным хешем (`password_hash`). Возвращает 1 при совпадении, 0 — при любой ошибке или несовпадении.

```
int db_check_credentials(const char *username, const char *password_hash) {
    PGconn *conn = db_connect();
    if (!conn) return 0;
```

```

const char *query = "SELECT password_hash FROM users WHERE username = $1";
const char *params[1] = { username };
PGresult *res = PQexecParams(conn, query, 1, NULL, params, NULL, NULL, 0);

if (PQresultStatus(res) != PGRES_TUPLES_OK) {
    fprintf(stderr, "Query failed: %s\n", PQerrorMessage(conn));
    PQclear(res);
    return 0;
}
if (PQntuples(res) == 0) {
    PQclear(res);
    return 0;
}
char *stored_hash = PQgetvalue(res, 0, 0);
int result = (strcmp(password_hash, stored_hash) == 0);
PQclear(res);
return result;
}

```

Взаимодействие с сервером/демоном:

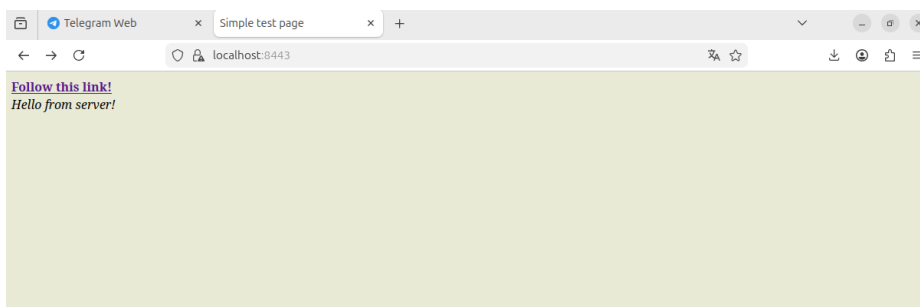
Модуль database.c является вспомогательным и не имеет собственной точки входа main(). Он предназначен для использования в составе демона аутентификации (auth_daemon.c). Демон, получив от сервера строку логин:хеш_пароля, вызывает db_check_credentials(). В зависимости от результата (1 или 0) демон формирует и отправляет серверу соответствующий ответ: SUCCESS, INVALID_PASSWORD или USER_NOT_FOUND.

Заголовочный файл database.h содержит объявления функций, позволяющие подключать этот модуль к другим частям проекта.

СОДЕРЖАНИЕ

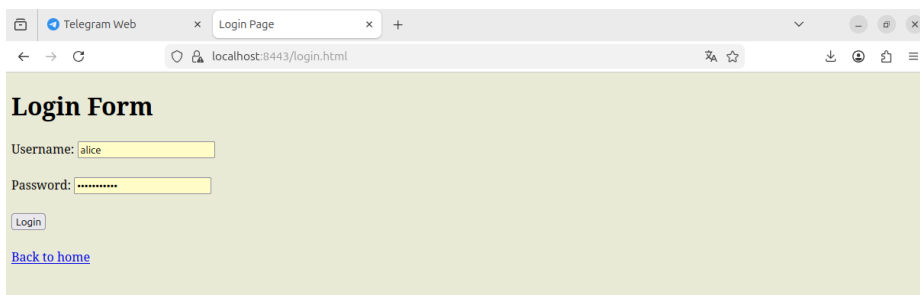
1. ЗАДАЧА	4
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
2.1. Введение	5
2.2. Общая архитектура системы	5
2.3. Алгоритм работы системы	5
2.4. Используемые библиотеки	6
2.5. Системные вызовы и функции	7
2.6. Функции, реализованные в программе	8
2.7. Алгоритм серверной части программы	9
2.8. Алгоритм работы модуля базы данных	10
2.9. Алгоритм работы демона аутентификации	10
2.10. Протокол взаимодействия и структуры данных	11
2. ПРАКТИЧЕСКАЯ ЧАСТЬ	12
3.1. Серверная часть Server.c	12
3.1.1. Вспомогательные функции	12
3.1.2. Создание серверного сокета	12
3.1.3. Настройка SSL/TLS	13
3.1.4. Парсинг данных аутентификации	14
3.1.5. Обработка запроса аутентификации	14
3.1.6. Отправка статических файлов	16
3.1.7. Обработка клиентского подключения	17
3.1.8. Основная функция	19
3.2. Демон аутентификации auth_daemon.c	20
3.2.1. Макрос аварийного завершения	20
3.2.2. Обработчик сигналов завершения	20
3.2.3. Демонизация процесса	20
3.2.4. Валидация SHA-256 хэша	21
3.2.5. Валидация имени пользователя	21
3.2.6. Обработка запроса аутентификации	22
3.2.7. Основной цикл демона	24
3.2.8. Точка входа в программу	27
3.3. Модуль базы данных database.h + database.c	27
3.3.1. Функция db_connect()	27
3.3.2. Функция db_check_credentials()	27
3.3.3. Взаимодействие с сервером/демоном	28
3. ПРИЛОЖЕНИЯ	30
Приложение 1. Главная страница веб-сайта (index.html)	30
Приложение 2. Страница формы аутентификации (login.html)	30
Приложение 3. Результат работы: Успешная аутентификация	31
Приложение 4. Результат работы: Пользователь не найден	31
Приложение 5. Результат работы: Неверный пароль	31
Приложение 6. Блок-схема проекта	32
Приложение 7. Код проекта.....	33

Приложения



Приложение 1. Главная страница веб-сайта (index.html)

```
index.html
<!DOCTYPE html>
<html>
<head>
  <title>Simple test page</title>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
</head>
<body bgcolor="#E9EAD6">
  <b style="color:#00AF00"><a href="login.html">Follow this link!</a></b>
  <br>
  <i>Hello from server!</i>
</body>
</html>
```



Приложение 2. Страница формы аутентификации (login.html)

```
login.html
<!DOCTYPE html>
<html>
<head>
  <title>Login Page</title>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
</head>
<body bgcolor="#E9EAD6">
  <h1>Login Form</h1>
  <form method="POST" action="/auth">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
```

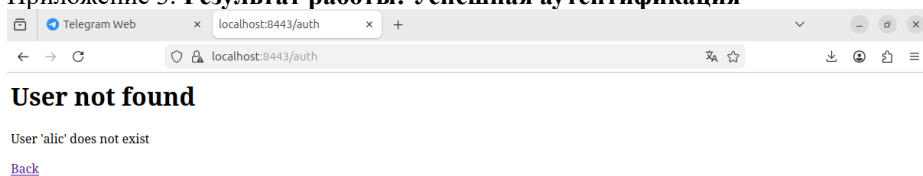
```

        <input type="submit" value="Login">
    </form>
    <br>
    <a href="index.html">Back to home</a>
</body>
</html>

```



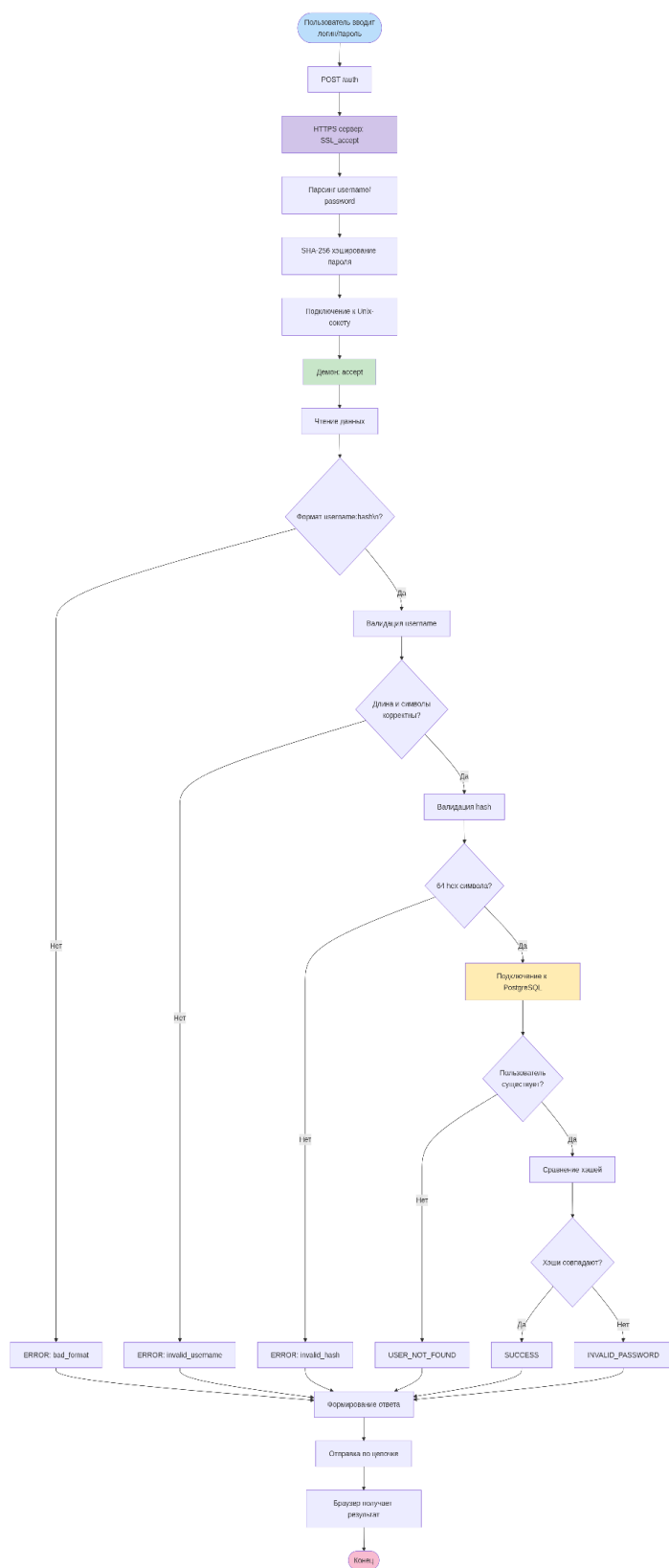
Приложение 3. Результат работы: Успешная аутентификация



Приложение 4. Результат работы: Пользователь не найден



Приложение 5. Результат работы: Неверный пароль



Приложение 6. Блок-схема проекта


```

server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <openssl/ssl.h>
#include <openssl/err.h>
#include <openssl/sha.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <fcntl.h>
#define GET "GET"
#define POST "POST"
#define HTTPS_HEADER "HTTP/1.0 200 OK\r\nContent-Type: text/html;charset=utf-8\r\n"
#define ERR404 "HTTP/1.1 404 Not Found\r\nContent-Type: text/html;charset=utf-8\r\n"
#define WROOT "./html"
#define INDEX "/index.html"
#define H404 "/h404.html"

#define BUFFER_SIZE 4096
#define SERVER_PORT 8443

#define MAX_USERNAME 64
#define MAX_PASSWORD 64
#define UNIX_SOCKET_PATH "/tmp/auth_daemon.sock"

void errExit(const char* msg);
int create_server_socket(int port);
SSL_CTX* create_ssl_context();
void configure_ssl_context(SSL_CTX* ctx);
void handle_client(int client_sock, SSL_CTX* ctx);
int parse_auth_data(const char* post_data, char* username, char* password);
void sha256_hash(const char* input, char* output);

void errExit(const char* msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

void sha256_hash(const char* input, char* output) {
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)input, strlen(input), hash);
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++) {
        sprintf(output + i * 2, "%02x", hash[i]);
    }
    output[SHA256_DIGEST_LENGTH * 2] = '\0';
}

int create_server_socket(int port) {
    int server_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (server_sock < 0) errExit("socket");

    int opt = 1;
    setsockopt(server_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;

```

```

memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(port);

if (bind(server_sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
    close(server_sock);
    errExit("bind");
}

if (listen(server_sock, 10) < 0) {
    close(server_sock);
    errExit("listen");
}

return server_sock;
}

SSL_CTX* create_ssl_context() {
    const SSL_METHOD* method = TLS_server_method();
    if (!method) {
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    SSL_CTX* ctx = SSL_CTX_new(method);
    if (!ctx) {
        fprintf(stderr, "Unable to create SSL context\n");
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    return ctx;
}

void configure_ssl_context(SSL_CTX* ctx) {
    if (SSL_CTX_use_certificate_file(ctx, "server.crt", SSL_FILETYPE_PEM) <= 0) {
        fprintf(stderr, "Failed to load certificate\n");
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    if (SSL_CTX_use_PrivateKey_file(ctx, "server.key", SSL_FILETYPE_PEM) <= 0) {
        fprintf(stderr, "Failed to load private key\n");
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    if (!SSL_CTX_check_private_key(ctx)) {
        fprintf(stderr, "Private key does not match certificate!\n");
        exit(EXIT_FAILURE);
    }

    printf("SSL certificate loaded successfully\n");
}

int parse_auth_data(const char* post_data, char* username, char* password) {
    char user_field[32], pass_field[32];

    if (sscanf(post_data, "%31[^]=%63[^&]&%31[^]=%63s",
        user_field, username,

```

```

    pass_field, password) != 4) {
        printf("Error parsing POST data: %s\n", post_data);
        return 0;
    }

    if (strcmp(user_field, "username") != 0 ||
        strcmp(pass_field, "password") != 0) {
        printf("Invalid field names in POST data\n");
        return 0;
    }

    for (char* p = username; *p; p++) if (*p == '+') *p = ' ';
    for (char* p = password; *p; p++) if (*p == '+') *p = ' ';

    printf("Parsing successful: user='%s', pass='%s'\n", username, password);
    return 1;
}

void handle_auth_request(SSL* ssl, char* post_data) {
    char username[MAX_USERNAME];
    char password[MAX_PASSWORD];
    char response[BUFFER_SIZE];

    printf("Authentication attempt\n");

    if (!parse_auth_data(post_data, username, password)) {
        snprintf(response, sizeof(response),
            "HTTP/1.1 400 Bad Request\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Content-Length: 55\r\n\r\n"
            "<h1>Data error</h1><a href='/'>Back</a>");
        SSL_write(ssl, response, strlen(response));
        return;
    }

    int sock;
    struct sockaddr_un addr;
    char request[256];
    char daemon_response[32];
    char password_hash[65];

    sha256_hash(password, password_hash);
    snprintf(request, sizeof(request), "%s:%s\n", username, password_hash);

    sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock == -1) {
        perror("[server] socket");
        goto auth_error;
    }

    struct timeval timeout;
    timeout.tv_sec = 5;
    timeout.tv_usec = 0;
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));
    setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, &timeout, sizeof(timeout));

    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, UNIX_SOCKET_PATH, sizeof(addr.sun_path) - 1);

    if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) == -1) {

```

```

    printf("[server] Cannot connect to auth daemon\n");
    close(sock);
    goto auth_error;
}

ssize_t sent_bytes = send(sock, request, strlen(request), 0);
if (sent_bytes == -1) {
    perror("[server] send to auth daemon");
    close(sock);
    goto auth_error;
}

memset(daemon_response, 0, sizeof(daemon_response));
ssize_t bytes = recv(sock, daemon_response, sizeof(daemon_response) - 1, 0);
close(sock);

if (bytes > 0) {
    daemon_response[bytes] = '\0';
    if (strcmp(daemon_response, "SUCCESS\n") == 0) {
        snprintf(response, sizeof(response),
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Content-Length: 45\r\n\r\n"
            "<h1>Login successful!</h1><a href='/'>Home</a>");
    }
    else if (strcmp(daemon_response, "INVALID_PASSWORD\n") == 0) {
        snprintf(response, sizeof(response),
            "HTTP/1.1 401 Unauthorized\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Content-Length: 65\r\n\r\n"
            "<h1>Invalid password</h1><p>Password is incorrect</p><a
href='/'>Back</a>");
    }
    else if (strcmp(daemon_response, "USER_NOT_FOUND\n") == 0) {
        snprintf(response, sizeof(response),
            "HTTP/1.1 401 Unauthorized\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Content-Length: 85\r\n\r\n"
            "<h1>User not found</h1><p>User '%s' does not exist</p><a
href='/'>Back</a>", username);
    }
    else {
        snprintf(response, sizeof(response),
            "HTTP/1.1 401 Unauthorized\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Content-Length: 55\r\n\r\n"
            "<h1>Authentication failed</h1><a href='/'>Back</a>");
    }
}
else {
    goto auth_error;
}

SSL_write(ssl, response, strlen(response));
return;

auth_error:
    snprintf(response, sizeof(response),
        "HTTP/1.1 401 Unauthorized\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        "Content-Length: 55\r\n\r\n"

```

```

        "<h1>Authentication error</h1><a href='/'>Back</a>");
    SSL_write(ssl, response, strlen(response));
}

void serve_static_file(SSL* ssl, const char* path) {
    char full_path[512];
    snprintf(full_path, sizeof(full_path), "%s%s", WROOT, path);

    struct stat fs;
    if (stat(full_path, &fs) < 0) {
        char error_path[512];
        snprintf(error_path, sizeof(error_path), "%s%s", WROOT, H404);

        int fd = open(error_path, O_RDONLY);
        if (fd >= 0 && stat(error_path, &fs) == 0) {
            char headers[512];
            snprintf(headers, sizeof(headers),
                "%sContent-Length: %ld\r\n\r\n", ERR404, fs.st_size);

            SSL_write(ssl, headers, strlen(headers));
            void* fmap = mmap(NULL, fs.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
            if (fmap != (void*)-1) {
                SSL_write(ssl, fmap, fs.st_size);
                munmap(fmap, fs.st_size);
            }
            close(fd);
        }
        return;
    }

    int fd = open(full_path, O_RDONLY);
    if (fd >= 0) {
        char headers[512];
        snprintf(headers, sizeof(headers),
            "%sContent-Length: %ld\r\n\r\n", HTTPS_HEADER, fs.st_size);

        SSL_write(ssl, headers, strlen(headers));
        void* fmap = mmap(NULL, fs.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
        if (fmap != (void*)-1) {
            SSL_write(ssl, fmap, fs.st_size);
            munmap(fmap, fs.st_size);
        }
        close(fd);
        printf("File sent: %s (%ld bytes)\n", path, fs.st_size);
    }
}

void handle_client(int client_sock, SSL_CTX* ctx) {
    SSL* ssl = SSL_new(ctx);
    SSL_set_fd(ssl, client_sock);

    if (SSL_accept(ssl) <= 0) {
        ERR_print_errors_fp(stderr);
        SSL_free(ssl);
        close(client_sock);
        return;
    }

    char buffer[BUFFER_SIZE];
    int bytes = SSL_read(ssl, buffer, sizeof(buffer) - 1);
    if (bytes <= 0) {

```

```

        SSL_free(ssl);
        close(client_sock);
        return;
    }
    buffer[bytes] = '\0';

    char method[16], path[256];
    if (sscanf(buffer, "%15s %255s", method, path) != 2) {
        SSL_free(ssl);
        close(client_sock);
        return;
    }

    printf("%s %s\n", method, path);

    if (strcmp(method, GET) == 0) {
        if (strcmp(path, "/") == 0) {
            serve_static_file(ssl, INDEX);
        }
        else {
            serve_static_file(ssl, path);
        }
    }
    else if (strcmp(method, POST) == 0 && strcmp(path, "/auth") == 0) {
        char* body = strstr(buffer, "\r\n\r\n");
        if (body) {
            body += 4;
            handle_auth_request(ssl, body);
        }
        else {
            char response[] = "HTTP/1.1 400 Bad Request\r\n\r\n\r\n";
            SSL_write(ssl, response, strlen(response));
        }
    }
    else {
        char response[] = "HTTP/1.1 405 Method Not Allowed\r\n\r\n\r\n";
        SSL_write(ssl, response, strlen(response));
    }

    SSL_shutdown(ssl);
    SSL_free(ssl);
}

int main() {
    printf("Starting HTTPS server (port %d)\n", SERVER_PORT);
    printf("HTML directory: %s\n", WROOT);
    printf("SSL certificate: server.crt\n");
    printf("SSL private key: server.key\n");
    printf("Auth daemon socket: %s\n", UNIX_SOCKET_PATH);
    printf("Open: https://localhost:%d\n", SERVER_PORT);
    printf("\n");

    SSL_library_init();
    SSL_load_error_strings();
    OpenSSL_add_all_algorithms();

    SSL_CTX* ctx = create_ssl_context();
    configure_ssl_context(ctx);

    int server_sock = create_server_socket(SERVER_PORT);
    printf("Server ready for HTTPS connections\n\n");

```

```

while (1) {
    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);
    int client_sock = accept(server_sock, (struct sockaddr*)&client_addr,
&client_len);

    if (client_sock >= 0) {
        printf("New connection from %s\n", inet_ntoa(client_addr.sin_addr));
        handle_client(client_sock, ctx);
        close(client_sock);
        printf("Connection closed\n\n");
    }
}

SSL_CTX_free(ctx);
close(server_sock);
return 0;
}

```

auth_daemon.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <ctype.h>

#define errExit(msg) do { perror(msg); exit(EXIT_FAILURE); } while (0)

#include "database.h"

#define SOCKET_PATH "/tmp/auth_daemon.sock"
#define BUF_SIZE 256
#define MAX_CLIENTS 10
#define USERNAME_MAX 50
#define HASH_MAX 65
#define MIN_USERNAME_LEN 1
#define MIN_HASH_LEN 64

struct Client {
    int fd;
    char buf[BUF_SIZE];
    size_t buf_len;
};

static volatile sig_atomic_t stop_flag = 0;

static void term_handler(int sig) {
    (void)sig;
    stop_flag = 1;
}

static void skeleton_daemon()

```

```

{
    pid_t pid;

    pid = fork();
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS);

    if (setsid() < 0) exit(EXIT_FAILURE);

    signal(SIGCHLD, SIG_IGN);
    signal(SIGHUP, SIG_IGN);

    signal(SIGTERM, term_handler);
    signal(SIGINT, term_handler);
    signal(SIGPIPE, SIG_IGN);

    pid = fork();
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS);

    umask(0);
    chdir("/");

    int x;
    for (x = sysconf(_SC_OPEN_MAX); x >= 0; x--) {
        close(x);
    }
}

static int is_valid_sha256_hash(const char* str) {
    if (strlen(str) != MIN_HASH_LEN) return 0;

    for (int i = 0; i < MIN_HASH_LEN; i++) {
        if (!isxdigit(str[i])) return 0;
    }
    return 1;
}

static int is_valid_username(const char* str) {
    int len = strlen(str);
    if (len < MIN_USERNAME_LEN || len >= USERNAME_MAX) return 0;

    for (int i = 0; i < len; i++) {
        if (!isalnum(str[i]) && str[i] != '_' && str[i] != '-' &&
            str[i] != '@' && str[i] != '.') {
            return 0;
        }
    }
    return 1;
}

static void process_auth_request(struct Client* client)
{
    char username[USERNAME_MAX];
    char password_hash[HASH_MAX];
    char response[BUF_SIZE];
    int auth_result;

    char* colon = strchr(client->buf, ':');
    if (colon == NULL) {
        strcpy(response, "ERROR:bad_format\n");
    }
}

```



```

    goto send_response;
}

*colon = '\0';
size_t username_len = strlen(client->buf);

if (username_len == 0) {
    strcpy(response, "ERROR:empty_username\n");
    goto send_response;
}

if (username_len >= USERNAME_MAX) {
    strcpy(response, "ERROR:username_too_long\n");
    goto send_response;
}

if (!is_valid_username(client->buf)) {
    strcpy(response, "ERROR:invalid_username\n");
    goto send_response;
}

strncpy(username, client->buf, sizeof(username) - 1);
username[sizeof(username) - 1] = '\0';

char* hash_start = colon + 1;
char* newline = strchr(hash_start, '\n');
if (newline == NULL) {
    strcpy(response, "ERROR:no_newline\n");
    goto send_response;
}

size_t hash_len = newline - hash_start;

if (hash_len == 0) {
    strcpy(response, "ERROR:empty_hash\n");
    goto send_response;
}

if (hash_len >= HASH_MAX) {
    strcpy(response, "ERROR:hash_too_long\n");
    goto send_response;
}

if (hash_len < MIN_HASH_LEN) {
    strcpy(response, "ERROR:hash_too_short\n");
    goto send_response;
}

strncpy(password_hash, hash_start, hash_len);
password_hash[hash_len] = '\0';

if (!is_valid_sha256_hash(password_hash)) {
    strcpy(response, "ERROR:invalid_hash_format\n");
    goto send_response;
}

PGconn *conn = db_connect();
if (!conn) {
    strcpy(response, "ERROR:db_connection\n");
    goto send_response;
}

```

```

const char *check_user_query = "SELECT id FROM users WHERE username = $1";
const char *check_user_params[1] = { username };

PGresult *user_res = PQexecParams(conn, check_user_query, 1, NULL,
check_user_params, NULL, NULL, 0);

if (PQresultStatus(user_res) != PGRES_TUPLES_OK) {
    PQclear(user_res);
    strcpy(response, "ERROR:db_query\n");
    goto send_response;
}

if (PQntuples(user_res) == 0) {
    PQclear(user_res);
    strcpy(response, "USER_NOT_FOUND\n");
    goto send_response;
}

PQclear(user_res);

auth_result = db_check_credentials(username, password_hash);

if (auth_result) {
    strcpy(response, "SUCCESS\n");
}
else {
    strcpy(response, "INVALID_PASSWORD\n");
}

send_response:
    send(client->fd, response, strlen(response), 0);

    client->buf_len = 0;
    memset(client->buf, 0, sizeof(client->buf));
}

static void run_daemon()
{
    int server_fd, rc;
    socklen_t len;
    struct sockaddr_un self;
    struct Client clients[MAX_CLIENTS];
    struct pollfd pfd[MAX_CLIENTS + 1];

    memset(&self, 0, sizeof(self));
    memset(clients, 0, sizeof(clients));

    for (int i = 0; i < MAX_CLIENTS; i++) {
        pfd[i].fd = -1;
        pfd[i].events = 0;
        clients[i].fd = -1;
    }

    len = sizeof(self);

    unlink(SOCKET_PATH);
    if ((server_fd = socket(AF_UNIX, SOCK_STREAM, 0)) == -1)
        errExit("socket");

    self.sun_family = AF_UNIX;

```

```

strcpy(self.sun_path, SOCKET_PATH);

if ((rc = bind(server_fd, (struct sockaddr*)&self, len)) == -1)
    errExit("bind");

if ((listen(server_fd, 10)) == -1)
    errExit("listen");

fcntl(server_fd, F_SETFL, fcntl(server_fd, F_GETFL) | O_NONBLOCK);

pfds[MAX_CLIENTS].fd = server_fd;
pfds[MAX_CLIENTS].events = POLLIN;

while (!stop_flag) {
    int ready = poll(pfds, MAX_CLIENTS + 1, 1000);

    if (ready == -1) {
        if (errno == EINTR) continue;
        errExit("poll");
    }

    if (ready == 0) continue;

    if (pfds[MAX_CLIENTS].revents & POLLIN) {
        int client_fd;
        struct sockaddr_un peer_addr;
        socklen_t peer_len = sizeof(peer_addr);

        client_fd = accept(server_fd, (struct sockaddr*)&peer_addr, &peer_len);
        if (client_fd == -1) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) continue;
            errExit("accept");
        }

        int slot = -1;
        for (int i = 0; i < MAX_CLIENTS; i++) {
            if (clients[i].fd == -1) {
                slot = i;
                break;
            }
        }

        if (slot != -1) {
            clients[slot].fd = client_fd;
            clients[slot].buf_len = 0;
            memset(clients[slot].buf, 0, sizeof(clients[slot].buf));

            fcntl(client_fd, F_SETFL, fcntl(client_fd, F_GETFL) | O_NONBLOCK);
            pfds[slot].fd = client_fd;
            pfds[slot].events = POLLIN;
        }
        else {
            close(client_fd);
        }
    }
}

for (int i = 0; i < MAX_CLIENTS; i++) {
    if (pfds[i].fd == -1) continue;

    if (pfds[i].revents & (POLLERR | POLLHUP)) {
        close(clients[i].fd);
    }
}

```

```

        clients[i].fd = -1;
        pfds[i].fd = -1;
        continue;
    }

    if (pfds[i].revents & POLLIN) {
        ssize_t bytes_read = recv(clients[i].fd,
            clients[i].buf + clients[i].buf_len,
            sizeof(clients[i].buf) - clients[i].buf_len - 1,
            0);

        if (bytes_read > 0) {
            clients[i].buf_len += bytes_read;
            clients[i].buf[clients[i].buf_len] = '\0';

            if (strchr(clients[i].buf, '\n') != NULL) {
                process_auth_request(&clients[i]);
            }
        }
        else if (bytes_read == 0) {
            close(clients[i].fd);
            clients[i].fd = -1;
            pfds[i].fd = -1;
        }
        else if (bytes_read == -1 && errno != EAGAIN && errno != EWOULDBLOCK) {
            close(clients[i].fd);
            clients[i].fd = -1;
            pfds[i].fd = -1;
        }
    }
}

for (int i = 0; i < MAX_CLIENTS; i++) {
    if (clients[i].fd != -1) close(clients[i].fd);
}

close(server_fd);
unlink(SOCKET_PATH);
}

int main(void)
{
    skeleton_daemon();
    run_daemon();
    return EXIT_SUCCESS;
}

database.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <postgresql/libpq-fe.h>

static PGconn *db_conn = NULL;

PGconn* db_connect() {
    if (db_conn != NULL && PQstatus(db_conn) == CONNECTION_OK) {
        return db_conn;
    }
}

```

```

const char *conninfo = "host=localhost port=5432 dbname=auth_system";

db_conn = PQconnectdb(conninfo);

if (PQstatus(db_conn) != CONNECTION_OK) {
    fprintf(stderr, "Database connection failed: %s\n", PQerrorMessage(db_conn));
    PQfinish(db_conn);
    db_conn = NULL;
    return NULL;
}
return db_conn;
}

int db_check_credentials(const char *username, const char *password_hash) {
    PGconn *conn = db_connect();
    if (!conn) {
        return 0;
    }

    const char *query = "SELECT password_hash FROM users WHERE username = $1";
    const char *params[1] = { username };

    PGresult *res = PQexecParams(conn, query, 1, NULL, params, NULL, NULL, 0);

    if (PQresultStatus(res) != PGRES_TUPLES_OK) {
        fprintf(stderr, "Query failed: %s\n", PQerrorMessage(conn));
        PQclear(res);
        return 0;
    }

    if (PQntuples(res) == 0) {
        PQclear(res);
        return 0;
    }

    char *stored_hash = PQgetvalue(res, 0, 0);
    int result = (strcmp(password_hash, stored_hash) == 0);
    PQclear(res);
    return result;
}

```

database.h

```

#ifndef DATABASE_H
#define DATABASE_H

#include <postgresql/libpq-fe.h>

PGconn* db_connect();
int db_check_credentials(const char* username, const char* password_hash);
int db_add_user(const char* username, const char* password_hash);

#endif

```

schema.sql

```

CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(64) NOT NULL, -- SHA-256 хэш (64 символа)
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
INSERT INTO users (username, password_hash) VALUES
('alice', 'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f'),
('bob', 'ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f')
ON CONFLICT (username) DO NOTHING;
```

Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -I/usr/include/postgresql
LDFLAGS = -lpq -lssl -lcrypto

all: auth_daemon server

database.o: database.c database.h
    $(CC) $(CFLAGS) -c database.c -o database.o

auth_daemon.o: auth_daemon.c database.h
    $(CC) $(CFLAGS) -c auth_daemon.c -o auth_daemon.o

server.o: server.c
    $(CC) $(CFLAGS) -c server.c -o server.o

auth_daemon: auth_daemon.o database.o
    $(CC) auth_daemon.o database.o -o auth_daemon $(LDFLAGS)

server: server.o
    $(CC) server.o -o server $(LDFLAGS)

clean:
    rm -f *.o auth_daemon server
    -rm /tmp/auth_daemon.sock

distclean: clean
    rm -f server.crt server.key

install_deps:
    sudo apt-get update
    sudo apt-get install -y gcc make libpq-dev libssl-dev postgresql postgresql-
contrib

certs:
    openssl req -x509 -newkey rsa:4096 -keyout server.key -out server.crt \
    -days 365 -nodes -subj "/C=US/ST=State/L=City/O=Org/CN=localhost"
    chmod 600 server.key

html_dir:
    mkdir -p html
    -cp -n *.html html/

.PHONY: all clean distclean html_dir certs install_deps
```